

Sequential Circuits

Alex Breuer

Overview and Schedule

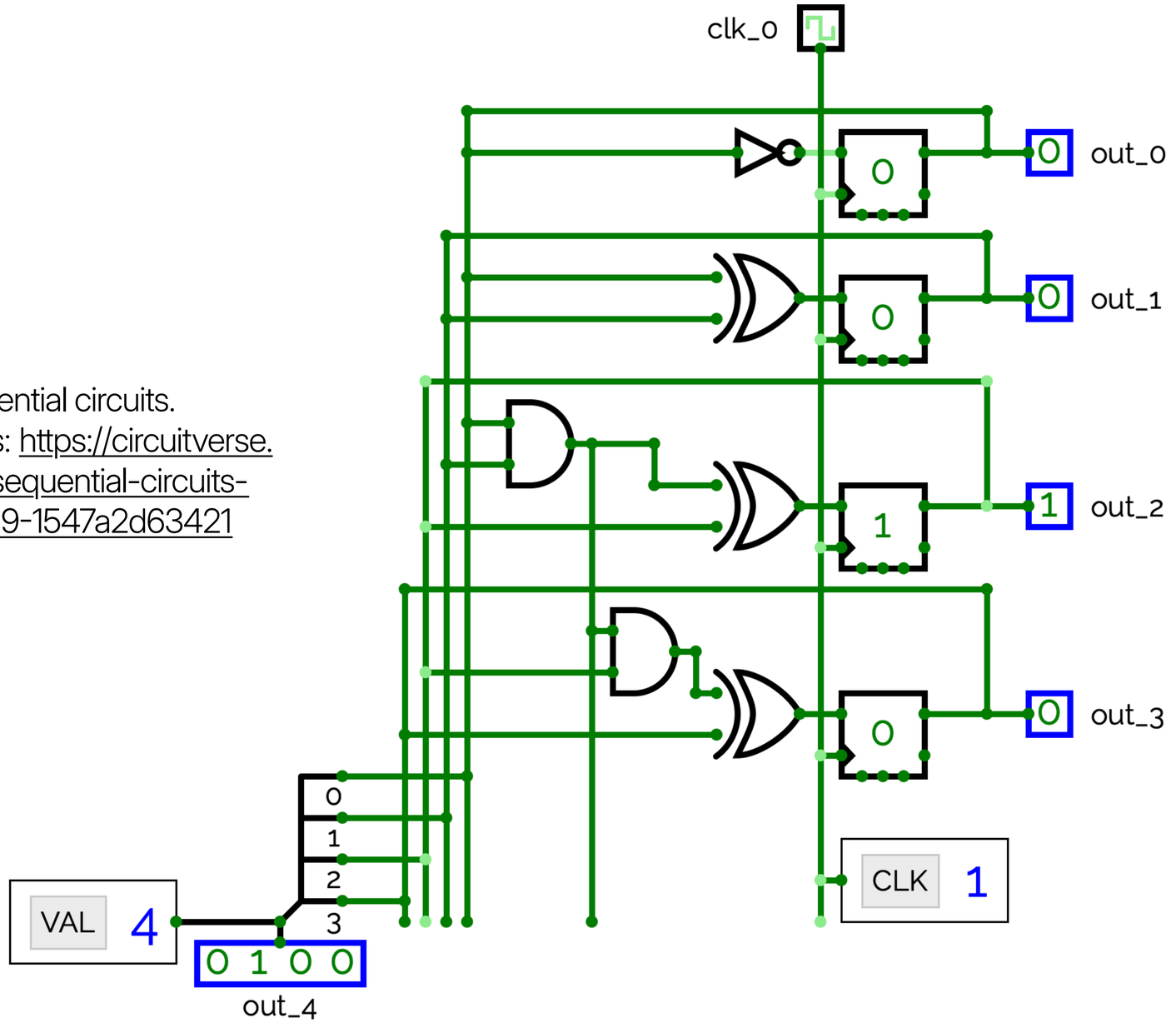
- Until this point: combinational circuits which don’t have any memory
- Now: (synchronous) sequential logic that has memory
- Roadmap:
  - Clock signals
  - Elementary building blocks: SR latch, D latch, D flip-flop, register
  - System timing
  - Pipelining
  - Example building blocks



SR Latch | D Latch | D Flip-Flop | Counter 0 | Counter 1 | Register File

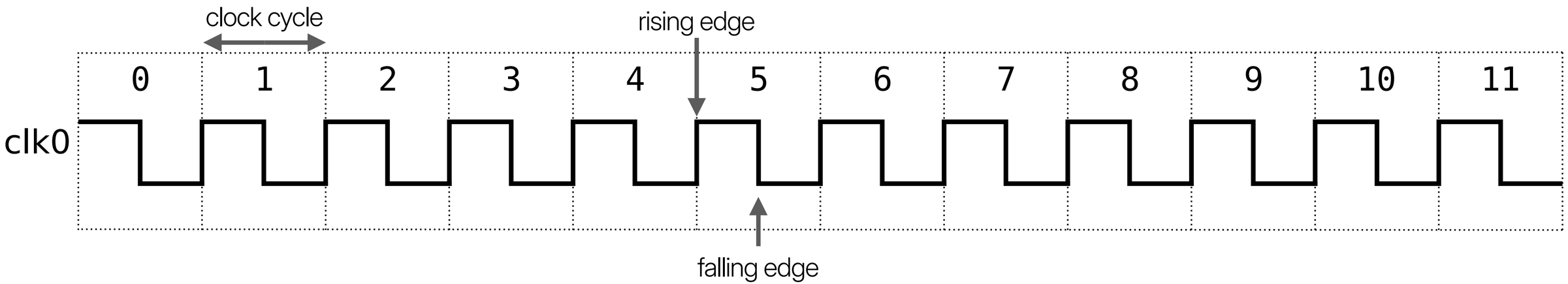
Full Screen
Time: 500
Clock:

We'll discuss different sequential circuits.
Interactive implementations: https://circuitverse.org/users/151679/projects/sequential-circuits-7667b241-baa3-47b4-bba9-1547a2d63421



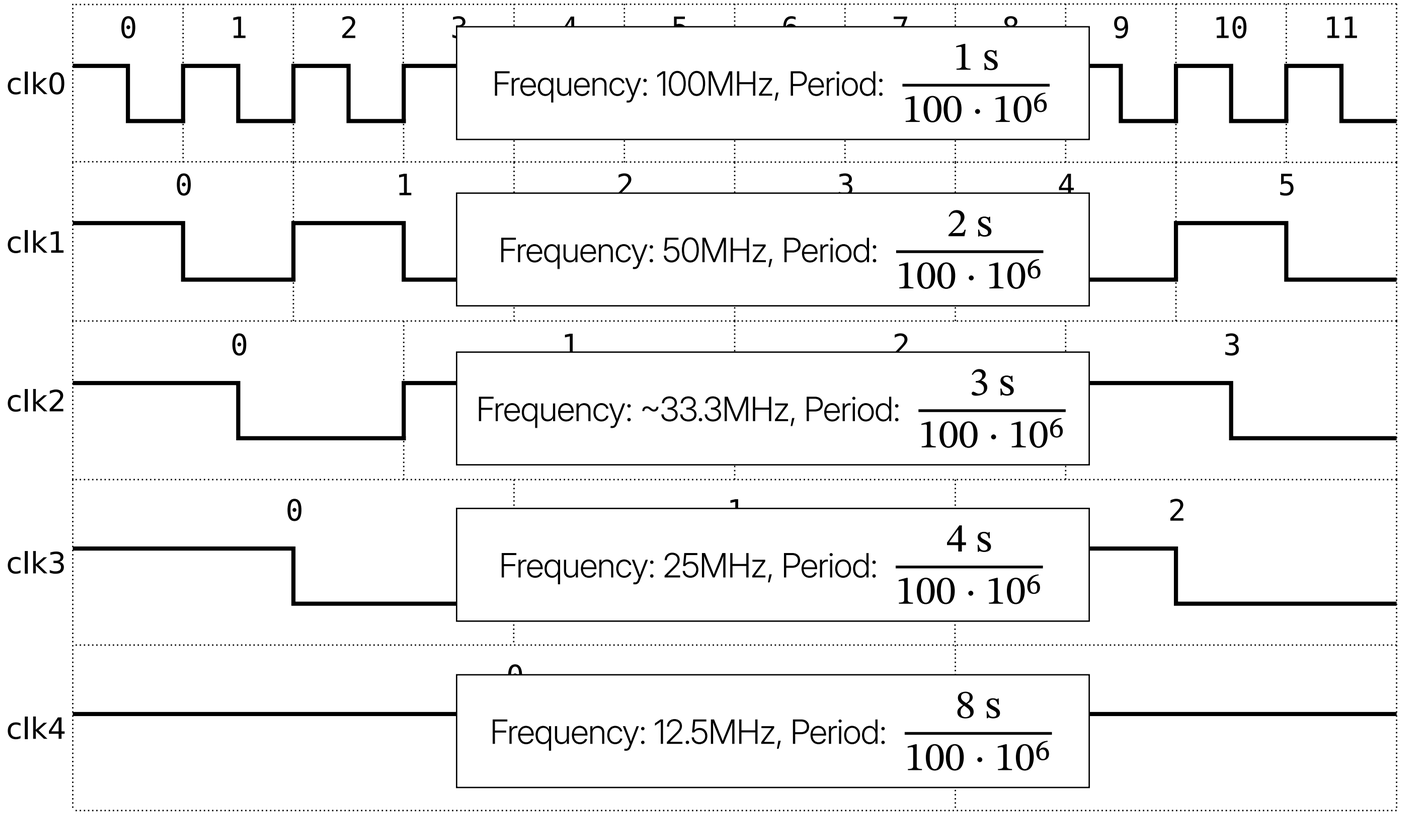
Clocks

Clock Signal



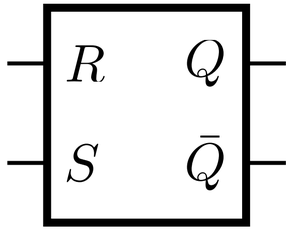
Frequency and Period

- Clock frequency: number of cycles per second
- Clock period: time per cycle
- Example on the right:
  - Assume that clk0 has a frequency of 100MHz
  - What's the clock period of clk0?
  - What are the clock frequencies and periods of clk1 - clk4?

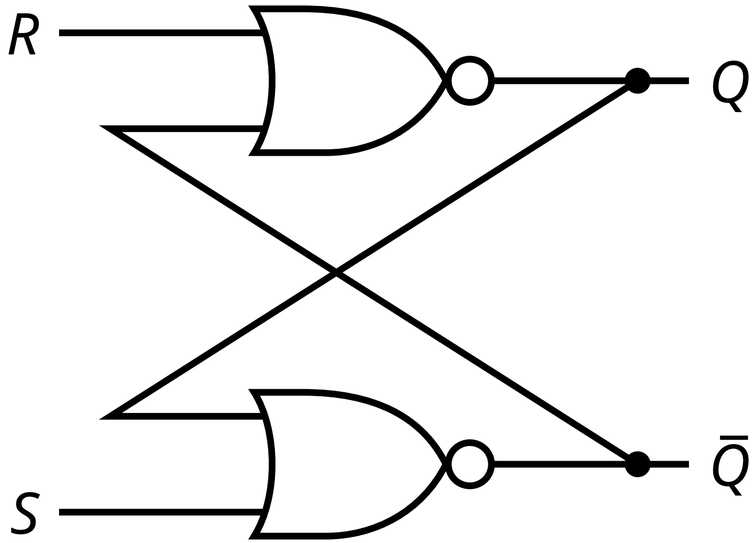


Latches and Flip-Flops

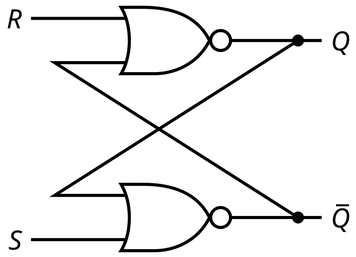
SR Latch

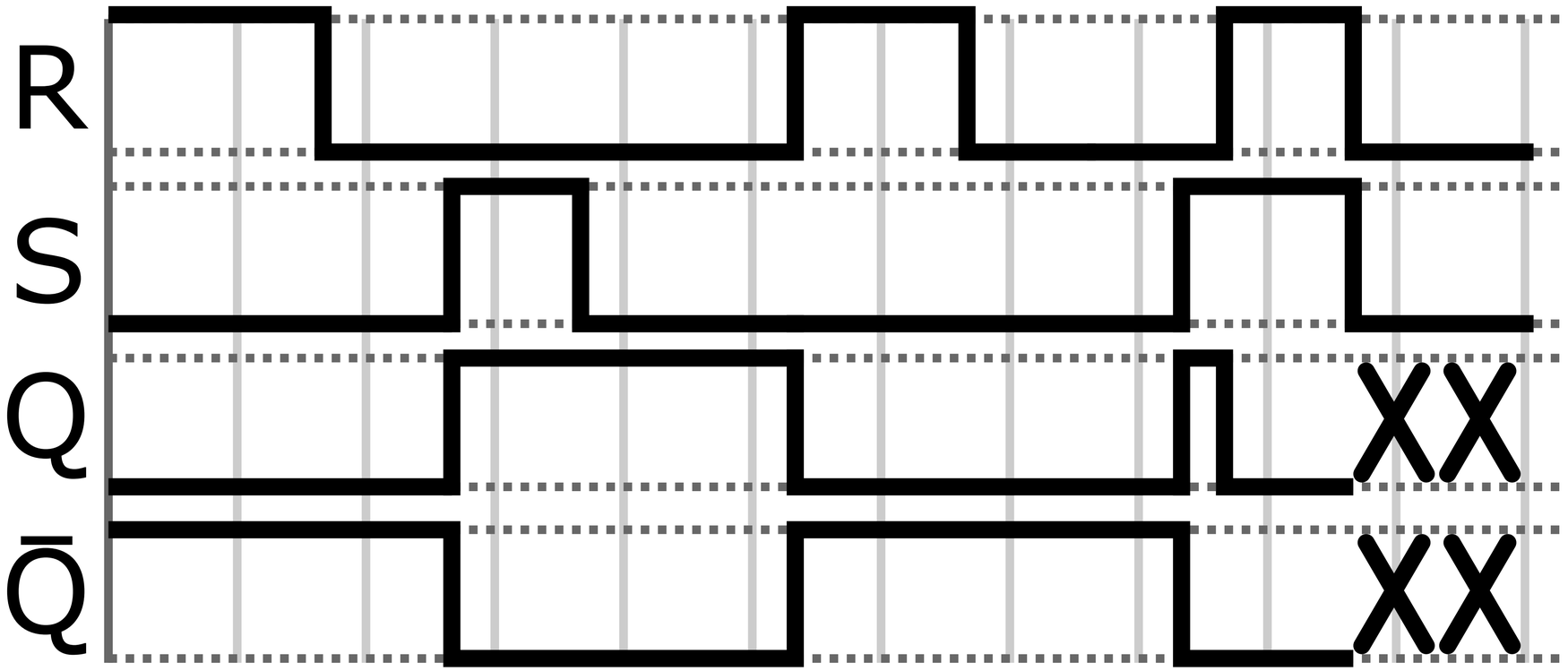


- Two inputs: S (Set) and R (Reset)
- Outputs: Q and $\bar{Q}$
- Possible implementation: two cross-coupled NOR gates
- If $R = S = 0$, the latch retains its stored state
- If only R is asserted, Q is reset to 0
- If only S is asserted, Q is set to 1
- If $R = S = 1$, both outputs are forced to 0. This is a forbidden input combination
- If R and S both change from 1 to 0 at the same time, the final state is unpredictable and metastability may occur



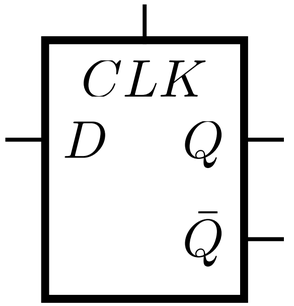
Example: SR Latch



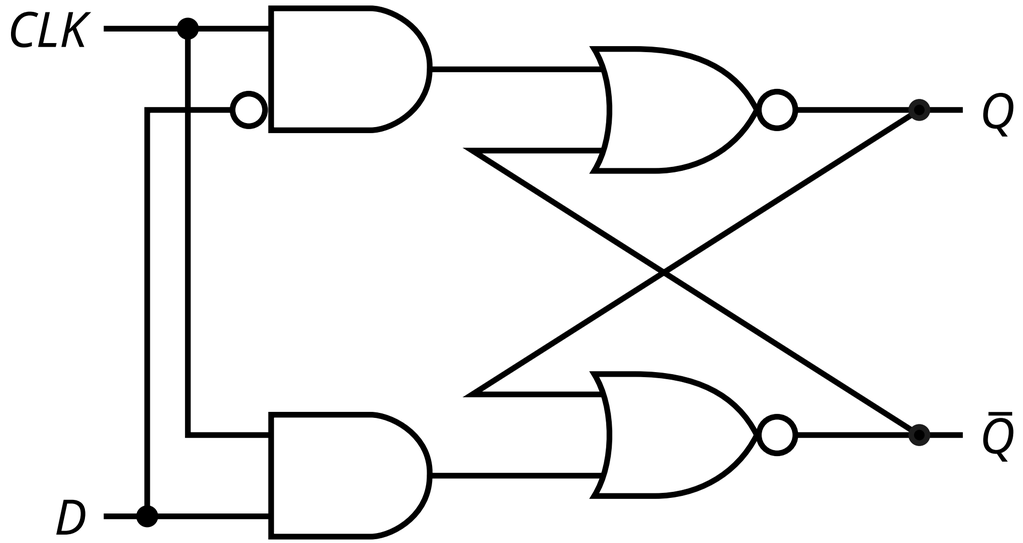


CircuitVerse — Simulation of an SR latch

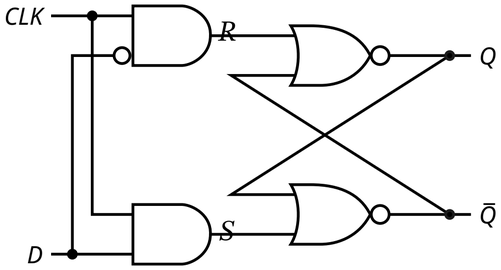
D Latch

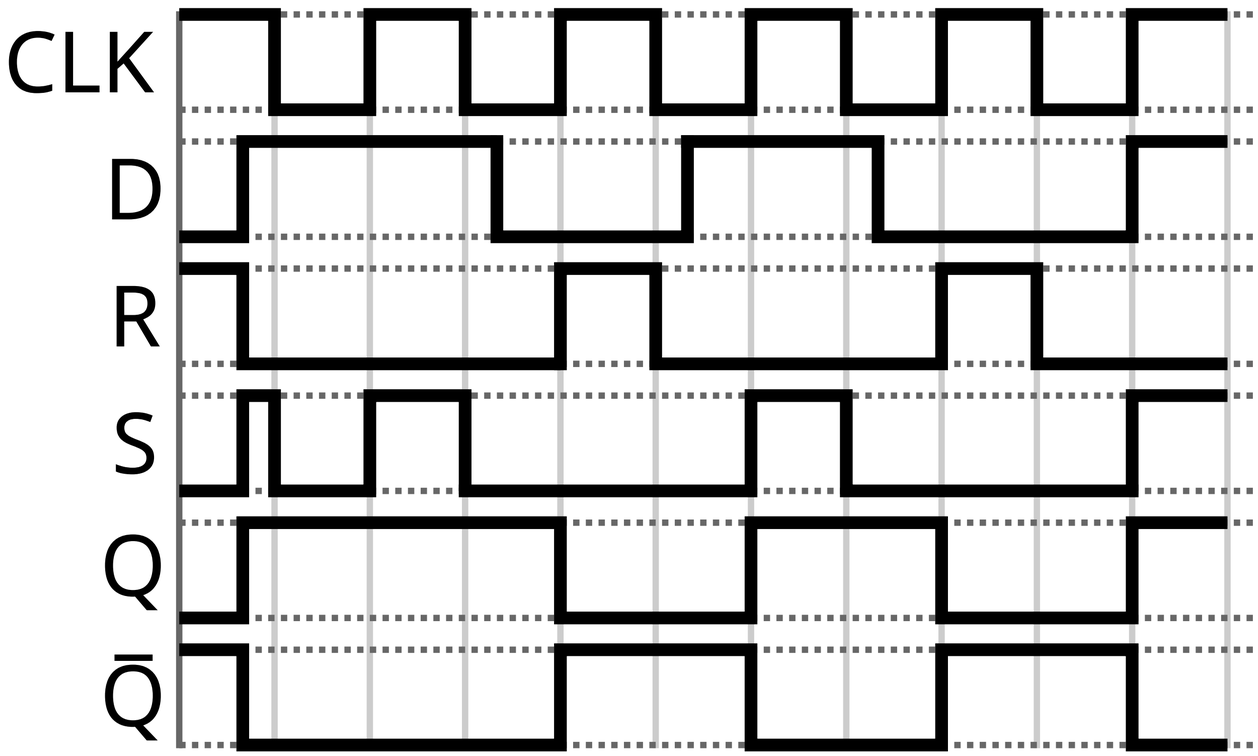


- Two inputs: CLK (clock signal) and D (data value)
- Outputs: Q and $\bar{Q}$
- Sets internal state Q to D when CLK is HIGH (latch is open / transparent)
- Remembers and returns the internal state Q when CLK is LOW (latch is closed / opaque)



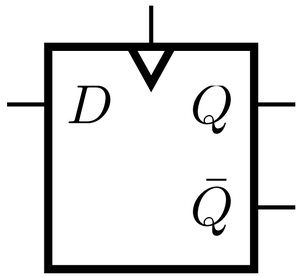
Example: D Latch



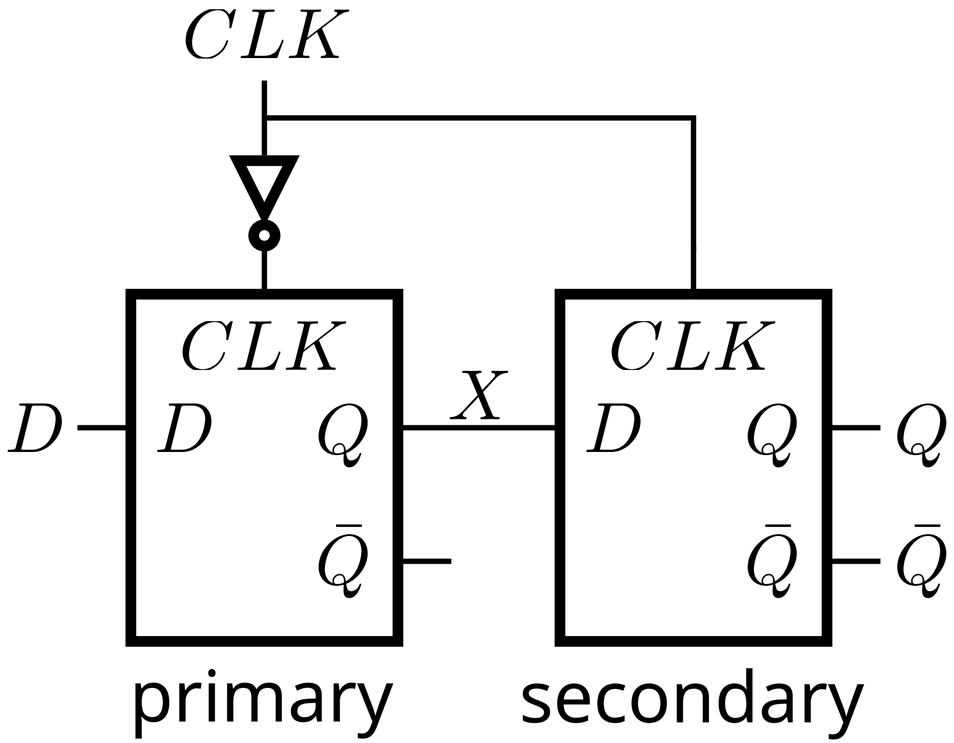


CircuitVerse — Simulation of a D latch

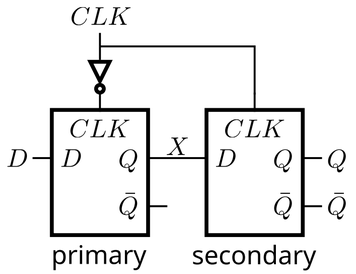
D Flip-Flop with Rising-edge Trigger

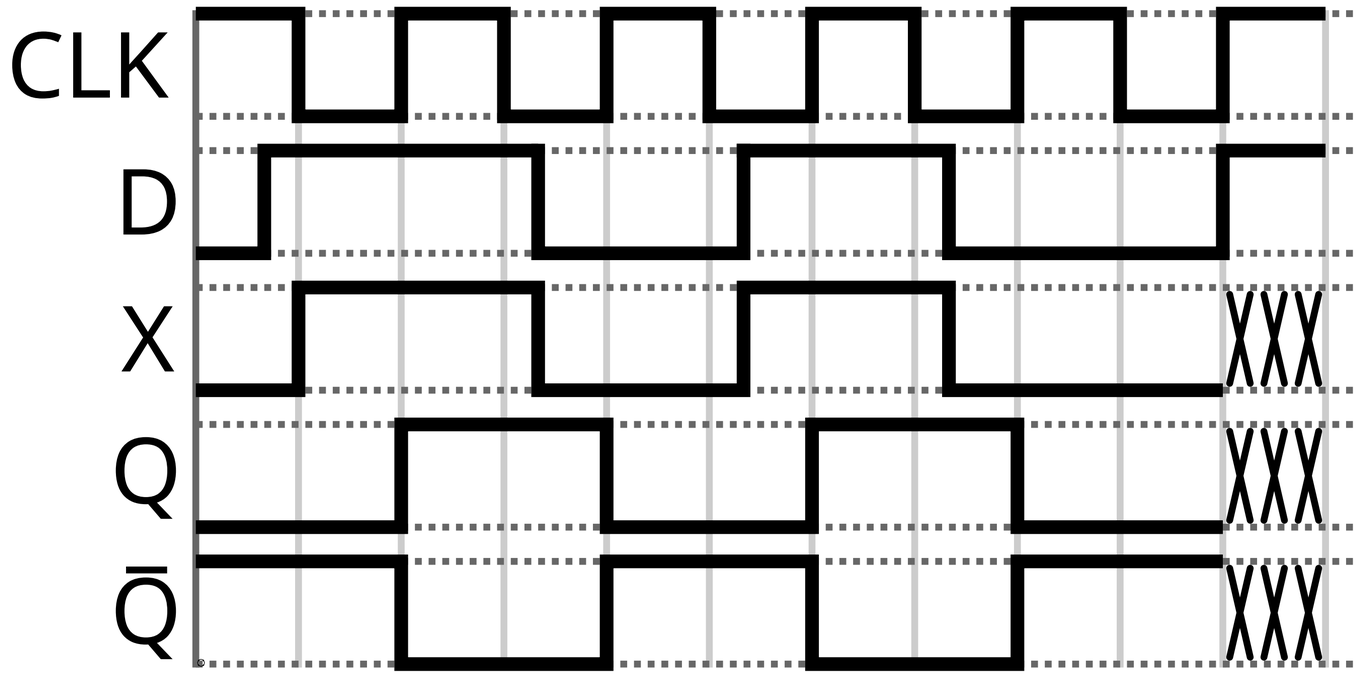


- Two inputs: "∇" (clock signal) and D (data value)
- Outputs: Q and $\bar{Q}$
- Sets internal state Q to D on rising edge of CLK
- Remembers and returns the internal state Q at all other times
- Built from two D latches (primary and secondary)



Example: D Flip-Flop

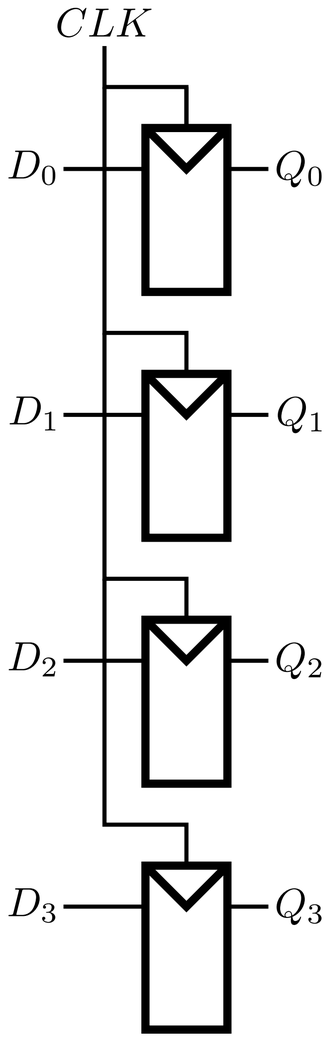




CircuitVerse — Simulation of a D flip-flop

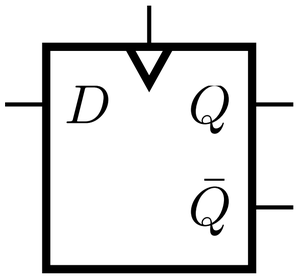
Register

- N flip-flops with a common clock build an N-bit register
- This means that the register is capable of storing N bits
- Example on the right: 4-bit register composed of four D flip-flops

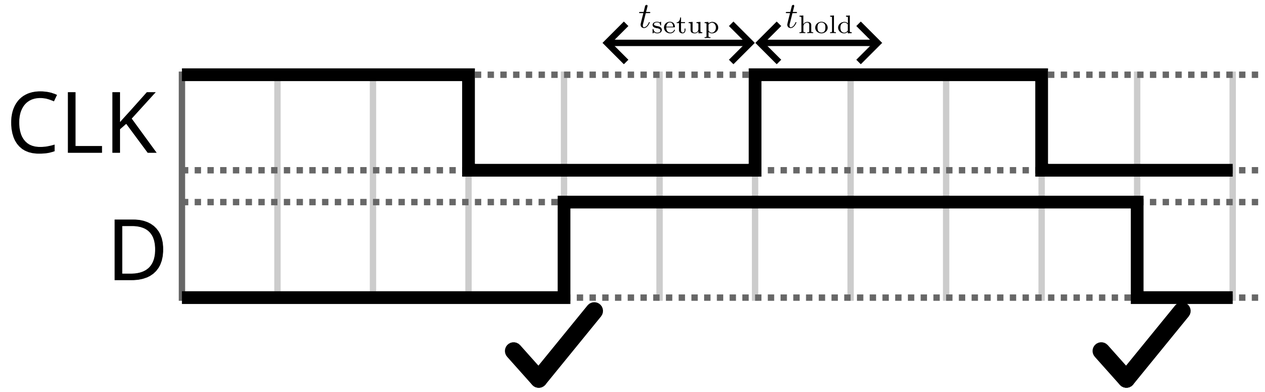


System Timing

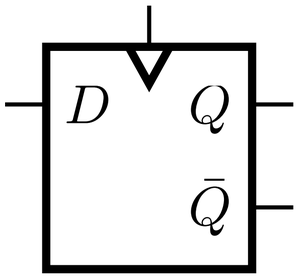
Setup and Hold Time



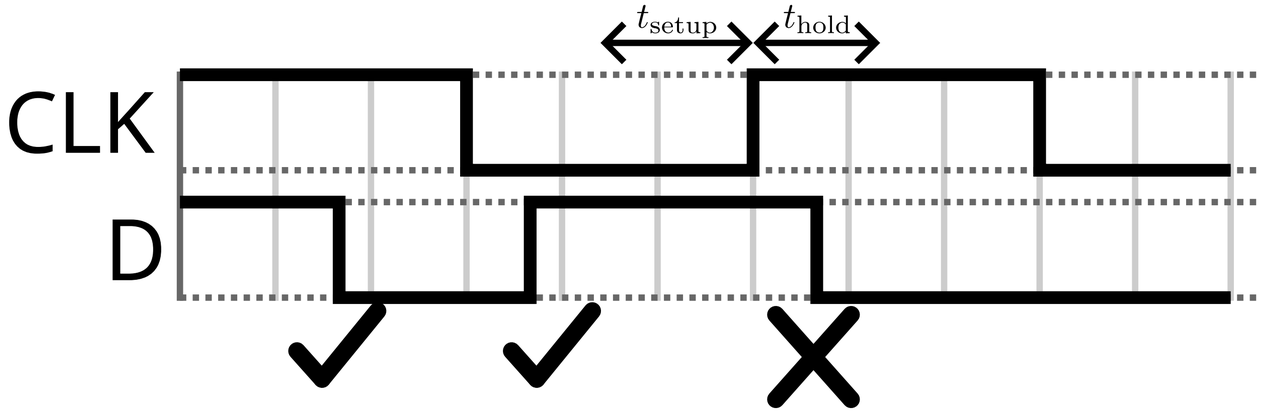
- We may not change the input of a flip-flop just around the triggering clock edge!
- Solution: Don't do it 🙄, i.e. provide stable input for a sufficient amount of time before (setup time) and after the clock edge (hold time)
- These requirements put limitations on our designs
- Example on the right: clock signal with respective setup time t_{setup} and hold time t_{hold} of a D flip-flop; does the input D meet the setup and hold requirements of the flip-flop?



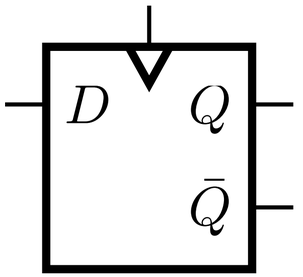
Setup and Hold Time



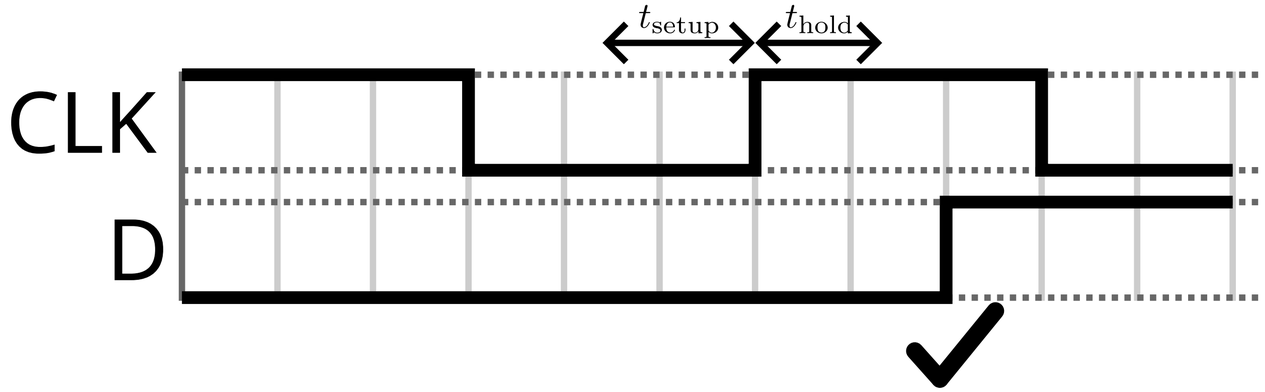
- We may not change the input of a flip-flop just around the triggering clock edge!
- Solution: Don't do it 🙄, i.e. provide stable input for a sufficient amount of time before (setup time) and after the clock edge (hold time)
- These requirements put limitations on our designs
- Example on the right: clock signal with respective setup time t_{setup} and hold time t_{hold} of a D flip-flop; does the input D meet the setup and hold requirements of the flip-flop?



Setup and Hold Time

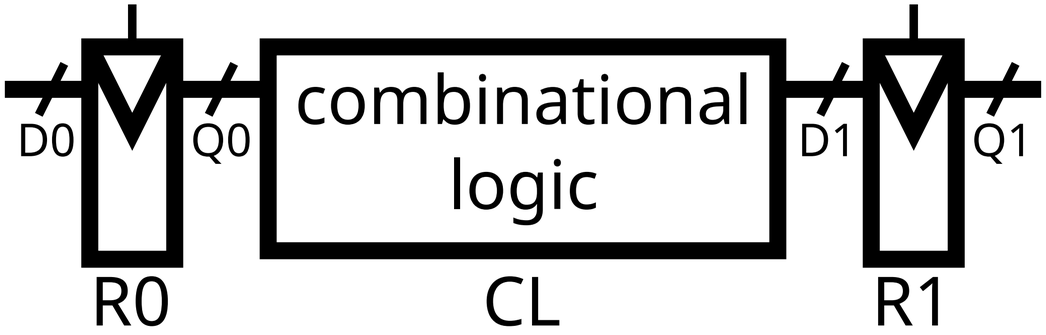


- We may not change the input of a flip-flop just around the triggering clock edge!
- Solution: Don't do it 🙄, i.e. provide stable input for a sufficient amount of time before (setup time) and after the clock edge (hold time)
- These requirements put limitations on our designs
- Example on the right: clock signal with respective setup time t_{setup} and hold time t_{hold} of a D flip-flop; does the input D meet the setup and hold requirements of the flip-flop?

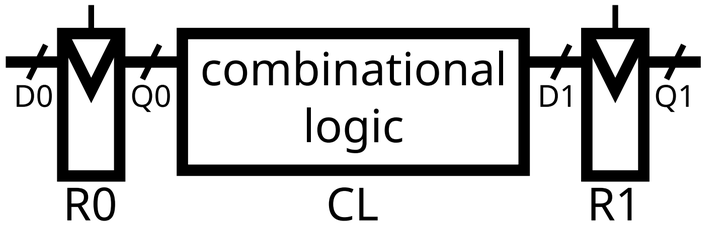


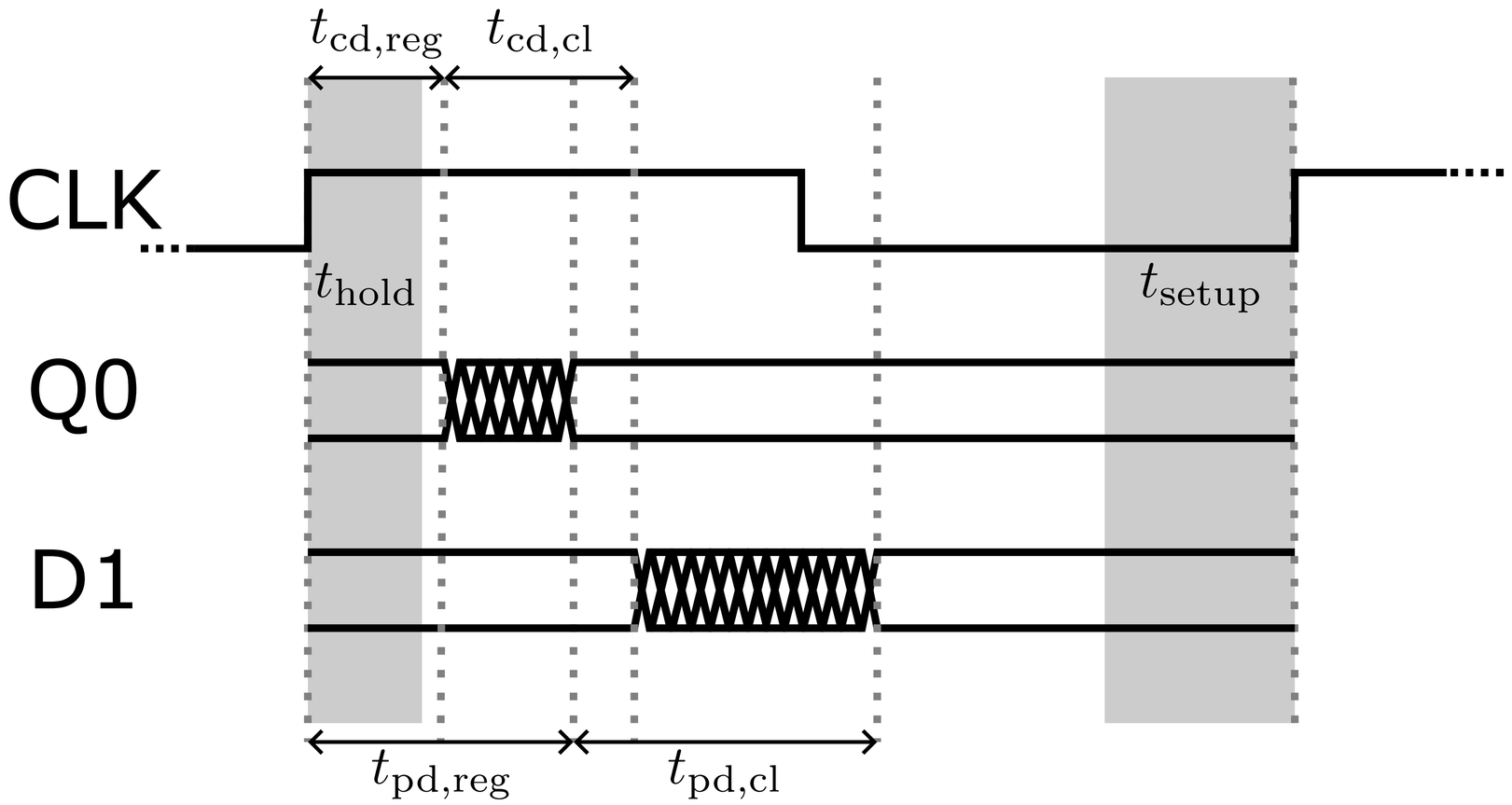
System Timing

- Assume two registers R0 and R1 that are connected by a piece of combinational logic CL
- Assume an ideal clock whose rising edges arrive at all registers simultaneously and occur exactly one clock period T_c apart
- We have to satisfy setup time and hold time constraints of R0 and R1
- Question: What are the limiting factors?
- Approach: Study respective delays within a clock cycle, i.e. between two consecutive rising edges. Derive respective requirements for R1



Contamination and Propagation Delays





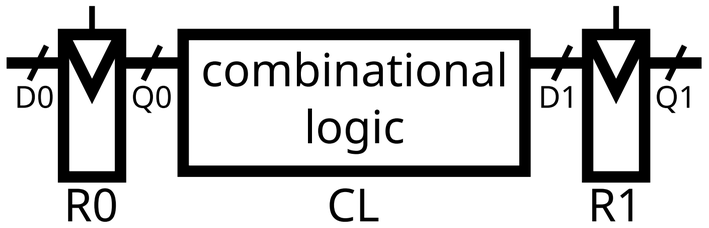
constraint of R1

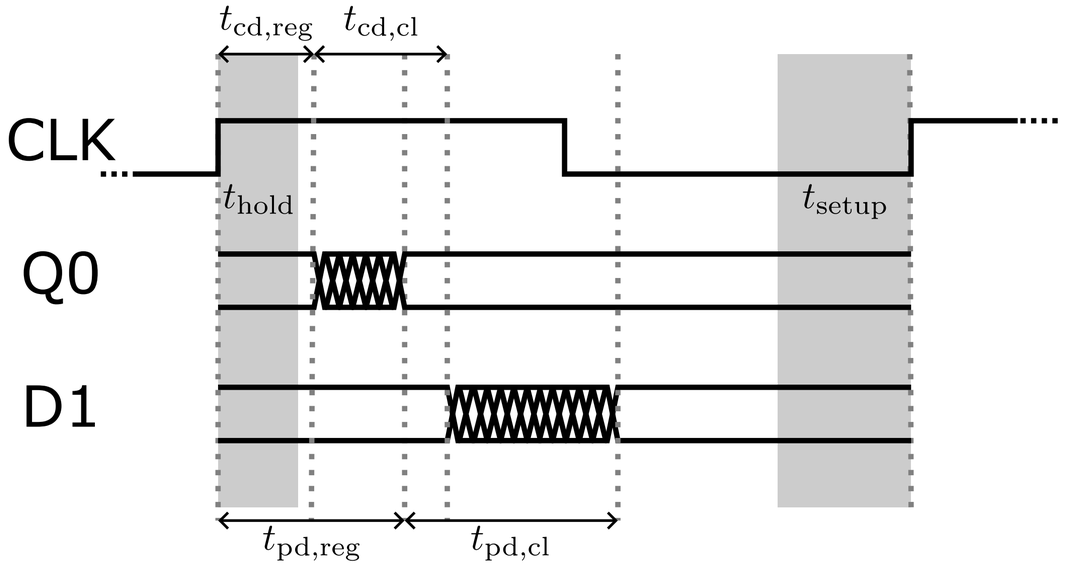
$$t_{cd,reg} + t_{cd,cl} \geq t_{hold}$$

$$t_{pd,reg} + t_{pd,cl} + t_{setup} \leq T_c$$ ← constraint of R1

System Timing: Observations

- Properties of registers are typically outside of our control, i.e. t_{hold}, t_{setup}, $t_{cd,reg}$ and $t_{pd,reg}$ are given
- If $t_{cd,reg} \geq t_{hold}$, the hold-time constraint is satisfied automatically. Otherwise, the combinational path must provide sufficient delay.
- Propagation delay of our combinational logic between registers limits frequency
- Approach for higher frequencies: shorten amount of time spent in combinational logic → pipelining



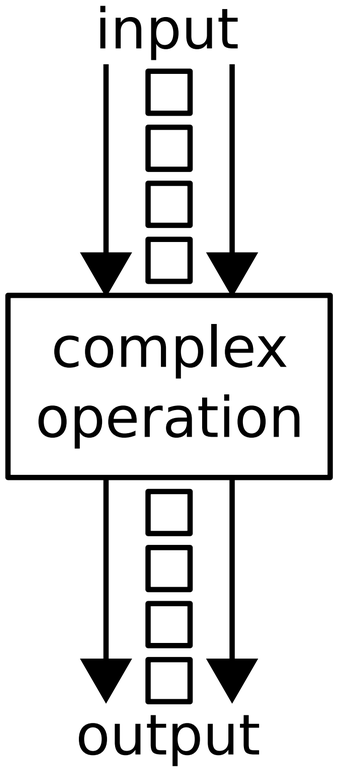


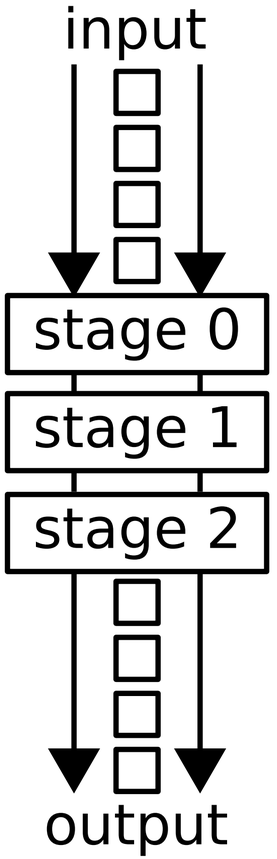
$$t_{cd,reg} + t_{cd,cl} \geq t_{hold}$$

$$t_{pd,reg} + t_{pd,cl} + t_{setup} \leq T_c$$

Pipelining

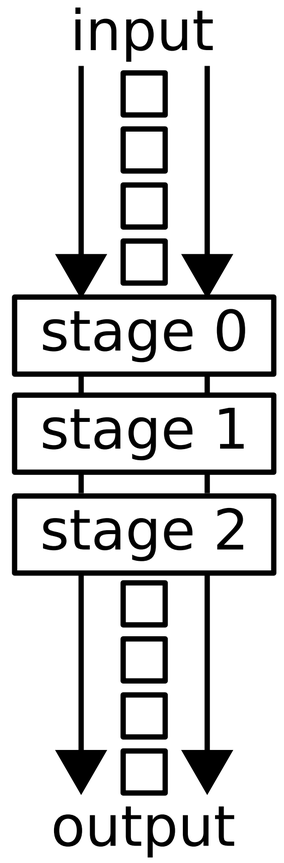
Pipelining


input
complex
operation
output


input
stage 0
stage 1
stage 2
output

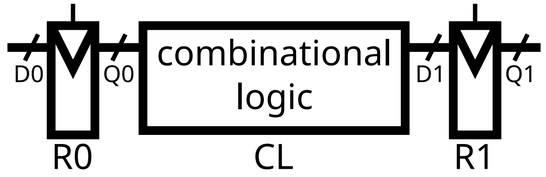
Pipelining

- Idea: split complex operation into several simpler (faster) stages
- Benefit: if we process the stages in parallel, we are able to increase the throughput
- Drawback: pipeline has to be filled to be efficient
- Instruction pipelining overlaps the processing of multiple instructions, exploiting instruction-level parallelism (ILP) → later in the course
- Pipelining is also applied at the “high-level”, e.g., for the parallelization of neural networks

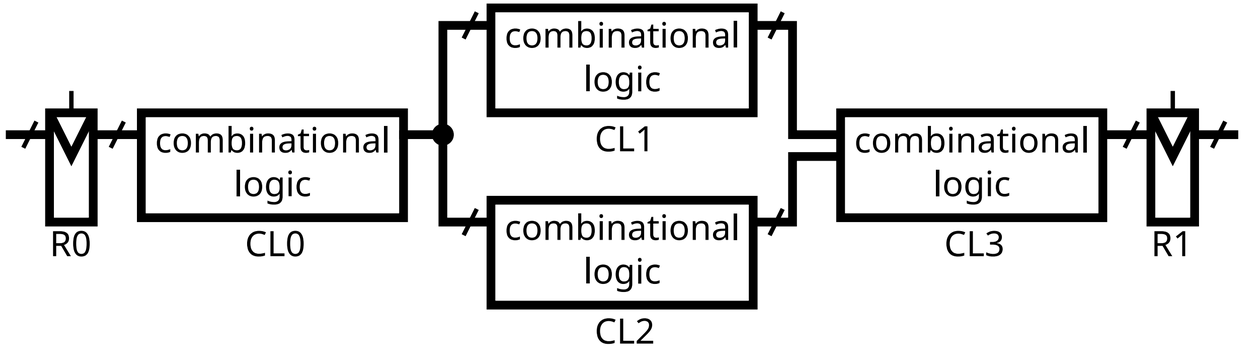


Pipelining through Intermediate Registers

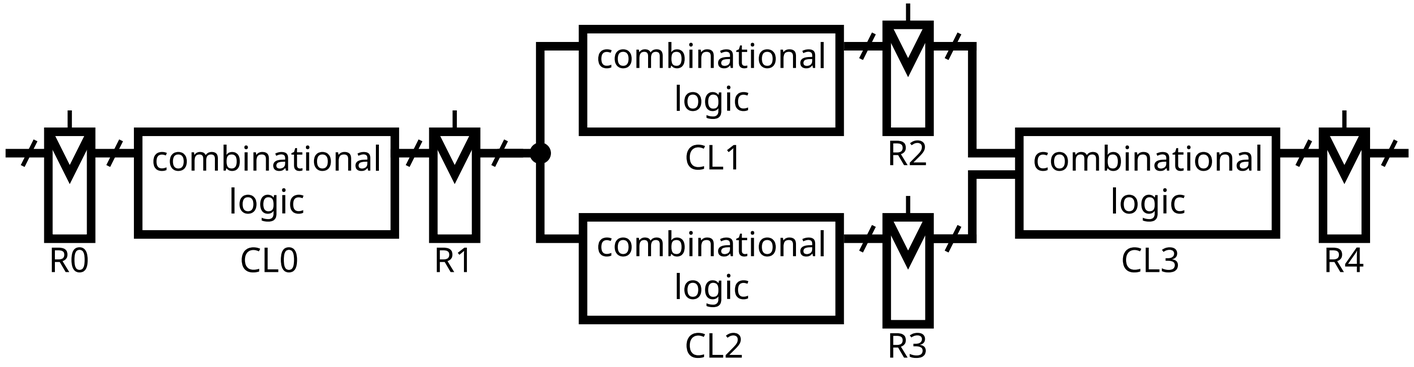
Given: block of "complex" combinational logic CL



Alternative formulation through smaller blocks



Pipelining: Add additional register so that the smaller blocks operate in parallel



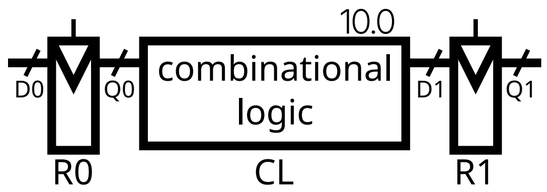
Pipelining: Speedup

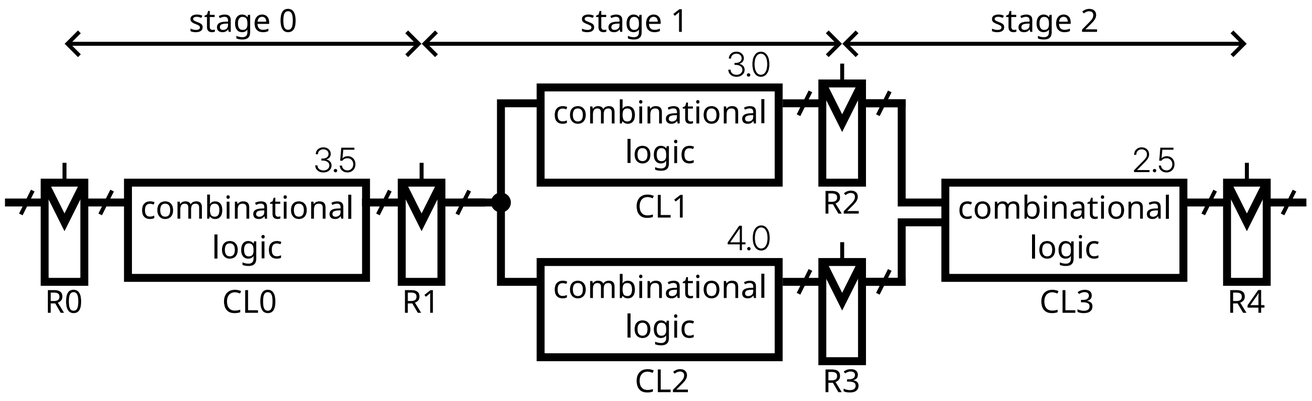
- Assumptions:
  - $t_{pd,reg} = 0.3, t_{setup} = 0.2, t_{pd,cl} = 10.0$
  - $(t_{pd,cl0}, t_{pd,cl1}, t_{pd,cl2}, t_{pd,cl3}) = (3.5, 3.0, 4.0, 2.5)$
- Constraint on clock period w/o pipelining

  $0.3 + 10.0 + 0.2 = 10.5 \leq T_c$
- Pipelined circuit bound by stage 1:

  $0.3 + 4.0 + 0.2 = 4.5 \leq T_c$
- Maximum throughput gain: $\frac{10.5}{4.5} = 2\frac{1}{3}$
- Latency then increases by a factor of $\frac{3 \cdot 4.5}{10.5} \approx 1.29$

$$t_{pd,reg} + t_{pd,cl} + t_{setup} \leq T_c$$

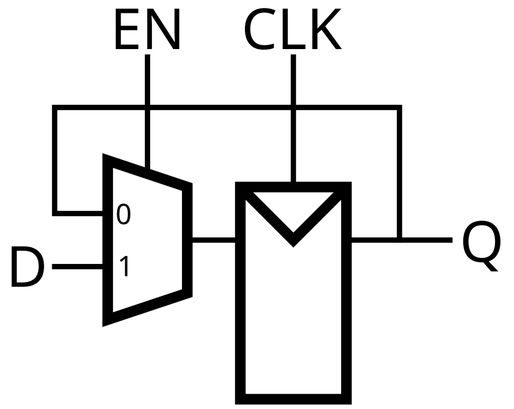




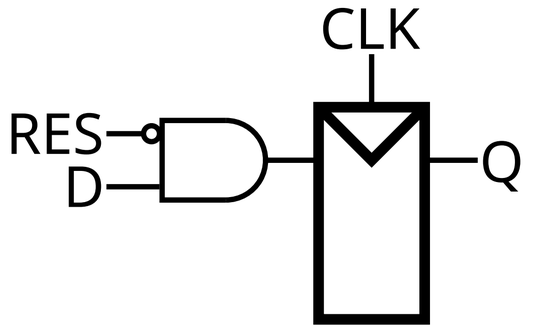
Common Sequential Circuits

Enabled and Resettable Flip-Flops

- Enabled flip-flop:
  - Sets internal state Q to D on rising edge of CLK if enable signal EN is asserted
  - Remembers and returns the internal state Q at all other times
- Resettable flip-flop:
  - Resets the flip-flop if the reset signal RES is asserted; either on rising edge (synchronous reset) or immediately (asynchronous reset)
  - Behaves like standard D flip-flop otherwise



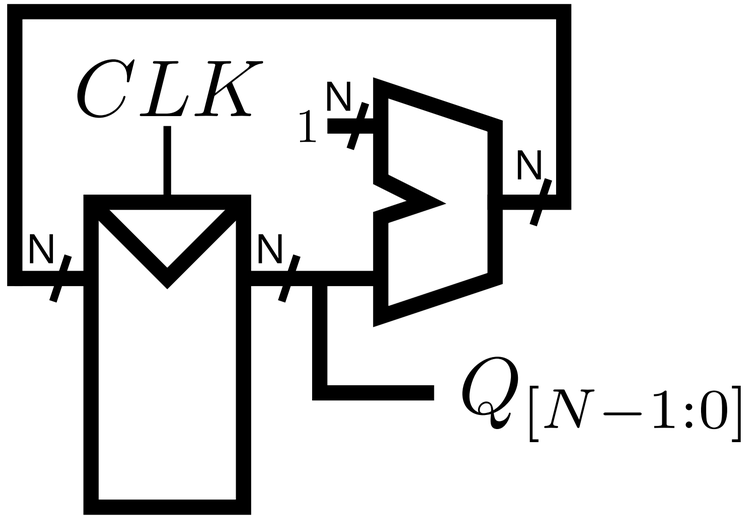
Possible implementation of an enabled flip-flop.



Possible implementation of a resettable flip-flop with synchronous reset.

Counters

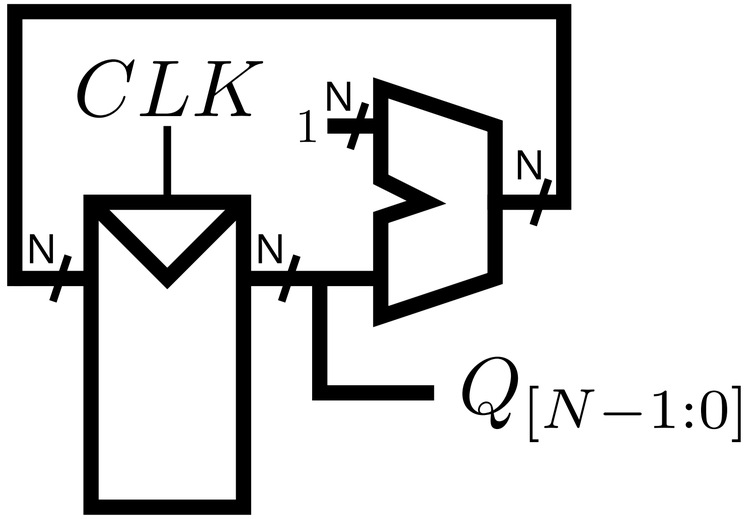
- N-bit binary counter counts from 0 to $2^N - 1$
- Increases the internal state on every rising edge
- Example on the right uses an adder to derive the next state
- Later in the course: the program counter contains the address of the instruction to be executed next. During sequential execution, it advances by the instruction length (4 bytes in A64).



Possible implementation of an N-bit binary counter.

Counters

- N-bit binary counter counts from 0 to $2^N - 1$
- Increases the internal state on every rising edge
- Example on the right uses an adder to derive the next state
- Later in the course: the program counter contains the address of the instruction to be executed next. During sequential execution, it advances by the instruction length (4 bytes in A64).

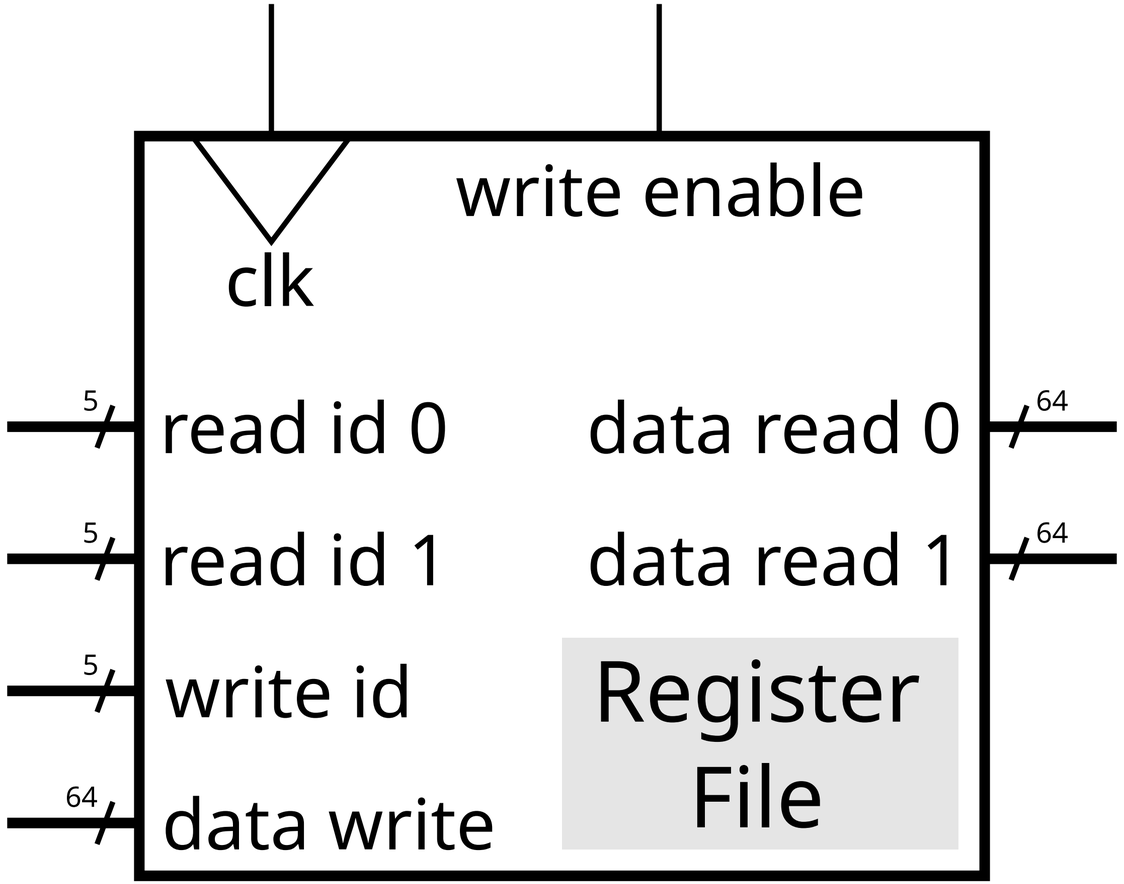


Possible implementation of an N-bit binary counter.

CircuitVerse — Implementation through adders & “simpler” implementation

Register File

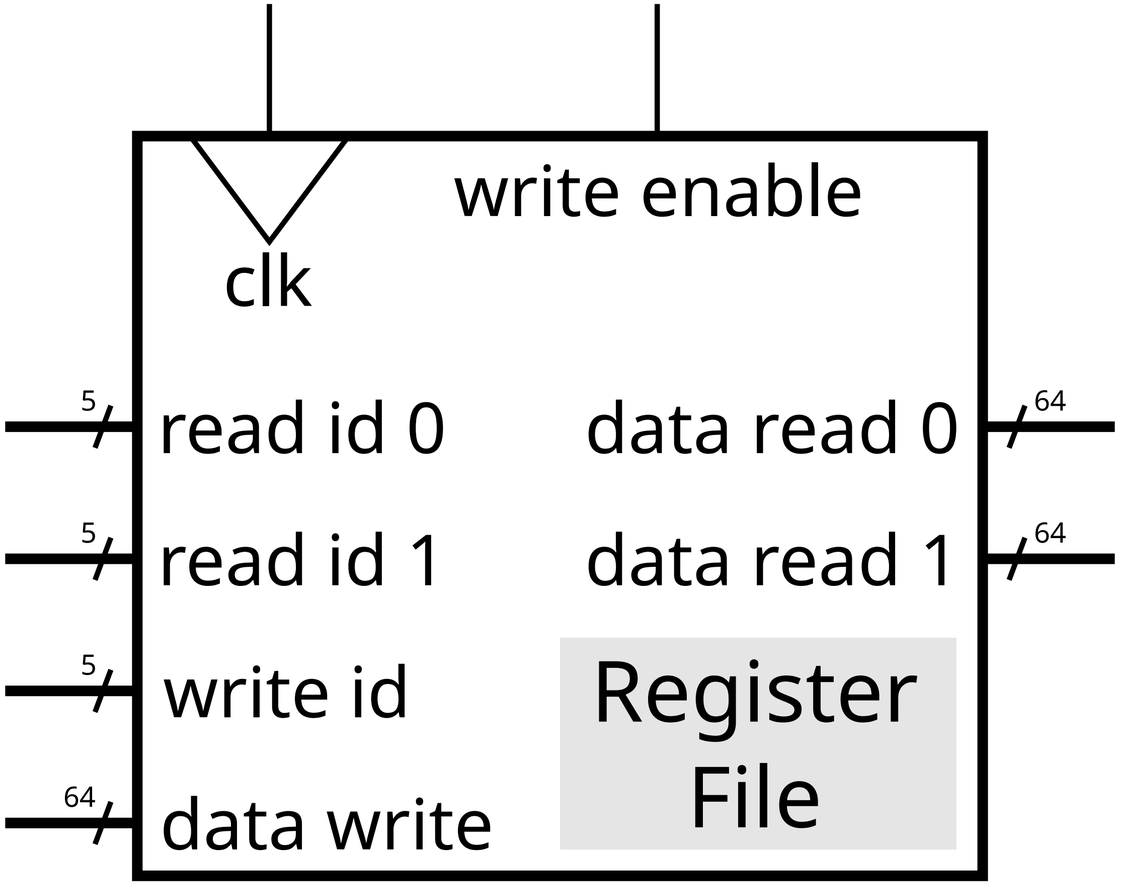
- Set of N-bit registers holds the internal state
- Structured way to read data from register file and write data to register file
- Inputs:
  - Ids of registers from which data is read
  - Id of register to which data is written
  - Data which is written
  - Clock signal
  - Write enable signal which decides if data is written on rising edge of clock
- Outputs:
  - Data of registers from which we are reading



Example of a register file with two read ports and a single write port. The register file comprises 32 64-bit registers.

Register File

- Set of N-bit registers holds the internal state
- Structured way to read data from register file and write data to register file
- Inputs:
  - Ids of registers from which data is read
  - Id of register to which data is written
  - Data which is written
  - Clock signal
  - Write enable signal which decides if data is written on rising edge of clock
- Outputs:
  - Data of registers from which we are reading



Example of a register file with two read ports and a single write port. The register file comprises 32 64-bit registers.

CircuitVerse — Simulation of a register file comprising four 4-bit registers.

Conclusions and Outlook

- Synchronous sequential logic is largely built from flip-flops
- The propagation delay of our combinational blocks limits the achievable clock frequency
- Pipelining allows us to increase the clock frequency / throughput at the cost of additional registers and higher latencies
- Discussed concepts are integral parts of modern processors:
  - Processors extensively rely on pipelining
  - Register file is key component of the data path
- Next: Assembly language

